ShortReel API Manual Testing Guide

Test by hand in Postman: pick the method, use the URL, press Send, read the result. No code. · Collection: **ShortReel manual tests** (`docs/postman/ShortReel-manual.postman_collection.json`) · 39 requests in 6 folders

Before you start

1. Import the collection: in Postman click **Import** and drop in `ShortReel-manual.postman_collection.json`.
2. Start a test server on port **3100** with the throwaway test database `shortreel_e2e_test`. Never use the real database. Setup steps are in `docs/postman/README.md`.
3. Check the server is up: send **Health check**. You should get `{"status":"ok","db":"up"}`.
4. Every URL starts with `http://localhost:3100/api`.
5. Go through folders 1 → 6 in order. Later requests use tokens and ids from earlier ones.

How to read the result

After pressing **Send**, look at two things: the **Status** (right side of the response, e.g. `200 OK`) and the **Body** (what the server sent back). Compare them with "Expected" in the request's description (the Documentation icon on the right). If they match, the test **passes**.

Status	Meaning
200 / 201	Success (201 = something new was created)
400	The data sent is invalid
401	Not logged in, or the token is wrong
402	Not enough coins
403	Logged in but not allowed
404	Not found
409	Duplicate, or conflicts with existing data

Important: a 401 or 403 is not always a failure. If "Expected" says 401 and you get 401, the test passes: it proves the server blocks people without access.

Lesson 1: No login needed

Open folder **1 No login needed**, click each request and press Send. All are GET, no body.

Request	Method	URL	Expected
Health check	GET	`/health`	200, status ok, db up
Film feed	GET	`/films`	200, approved films only
Film 1 details	GET	`/films/1`	200, includes the episode list
Free episode (episode 1)	GET	`/episodes/1`	200, has a `stream_url`

Paid episode (episode 3)	GET	`/episodes/3`	200, `stream_url` is null
Stream a paid episode without login	GET	`/episodes/3/stream`	401
Lookups	GET	`/lookups`	200
Coin packs	GET	`/coin-packs`	200

Lesson 2: Register and use a token

A **token** is the pass you get when you register or log in. Requests that need a login must carry it, otherwise the server answers 401.

Step 1: Register a viewer

1. Open folder **2 Register and login** and click **Register viewer**.
2. The **Body** tab (raw / JSON) is already filled in:

   ```
   { "username": "viewer_test", "password": "Sprint12pw", "role": "viewer" }
   ```

3. Press **Send**. You should get **201 Created** and a body with `"token": "eyJhbGciOi..."`.

Step 2: Save the token

1. Copy the `token` value, only what is inside the quotes, without the `"`.
2. In the left sidebar click the collection name, then open the **Variables** tab.
3. Paste it into the **Current value** of `token_viewer`.
4. Press **Save** (Ctrl+S).

Step 3: Check the token works

1. **Profile without token** → Send → should be **401**.
2. **Profile with token** → the **Authorization** tab shows Bearer Token = `{{token_viewer}}` → Send → should be **200** with `"username": "viewer_test"`.

> Still 401? Make sure you pressed Save and did not copy the `"` characters.

Step 4: Cases that should be refused

Request	Body	Expected	Meaning
Register duplicate username	`{"username":"viewer_test","password":"Sprint12pw"}`	409	Username already taken
Password too short	`{"username":"short_test","password":"a1"}`	400	Password fails the rules
Register as admin (refused)	`{..., "role":"admin"}`	403	Nobody can sign up as admin
Login wrong password	`{"username":"viewer_test","password":"Wrong123pw"}`	401	Wrong password
Login viewer	`{"username":"viewer_test","password":"Sprint12pw"}`	200	Gives a fresh token

Step 5: Save two more tokens

- **Register creator** (expect 201) → copy the token into `token_creator`
- **Login admin** (expect 200) → copy the token into `token_admin`

> Each username can be registered only once; a second time gives 409. If you lose a token, use the **Login** request instead of registering again.

Lesson 3: Coins and unlocking (uses token_viewer)

Go in this order, because the coin balance carries over. Episode 3 costs 20 coins.

#	Request	Method / URL	Body	Expected
1	Wallet balance	GET `/wallet`		200, balance 0
2	Unlock with no coins	POST `/episodes/3/unlock`		402, not enough coins
3	Top up demo +100	POST `/wallet/topup`	`{"demo":true}`	201, 100 coins (simulated)
4	Unlock episode 3	POST `/episodes/3/unlock`		200, charged 20, balance 80
5	Unlock again	POST `/episodes/3/unlock`		200, not charged again, still 80
6	Episode 3 after unlock	GET `/episodes/3`		200, now has a `stream_url`
7	Coin history	GET `/wallet/transactions`		200, one top-up and one unlock

What this proves: no coins means no purchase, the server sets the price, buying twice does not charge twice, and a bought episode becomes playable.

Lesson 4: Subscription (uses token_viewer)

Request	Method / URL	Expected
Subscription status	GET `/subscription`	200, not subscribed yet
Subscribe	POST `/subscription`	201, active, with start and end dates
Cancel subscription	DELETE `/subscription`	200, still active until the end date

Lesson 5: Creator films and upload

Request	Token	Method / URL	Body	Expected
Viewer cannot create a film	viewer	POST `/films`	`{"title":"x","genre":"Drama","ai_tool":"Mixed","is_ai_generated":true}`	403
Create film without AI label	creator	POST `/films`	`{..., "is_ai_generated":false}`	400
Create film	creator	POST `/films`	`{"title":"Test film","genre":"Drama","ai_tool":"Mixed","prompt_credit":"prompt credit","is_ai_generated":true}`	201, status pending
Upload episode (pick an mp4 file)	creator	POST `/films/{{film_id}}/episodes`	form-data (see below)	201

Save film_id

After **Create film** returns 201, copy the `"id"` value (a number, e.g. `6`) into the `film_id` variable in the Variables tab and press Save.

Upload a video file

1. Click **Upload episode (pick an mp4 file)**, open **Body** → **form-data**.
2. In the `video` row (type File) click **Select Files** and choose an mp4, for example one in `backend/uploads`.
3. Keep `price_coins` = `0` (free) and `episode_no` = `1`.
4. Press Send. You should get **201**.

> Try `price_coins` = 501: you should get 400 (the maximum is 500). Try a file that is not a video: you should get 400.

Lesson 6: Admin

Request	Token	Method / URL	Body	Expected
Viewer cannot open the review queue	viewer	GET `/admin/reviews`		403
Review queue	admin	GET `/admin/reviews`		200, lists pending films
Approve film	admin	POST `/admin/films/{{film_id}}/review`	`{"status":"approved"}`	200, approved
Reject without a note	admin	POST `/admin/films/4/review`	`{"status":"rejected"}`	400, a note is required
Revenue share	admin	GET `/admin/settings/revenue-share`		200
Set revenue share to 101%	admin	PUT `/admin/settings/revenue-share`	`{"admin_share_percent":101}`	400
Sales	admin	GET `/admin/sales`		200

> After approving, go back to **Film feed** in folder 1: "Test film" now appears in the feed.

Common problems

Symptom	Cause / fix
Could not get response / ECONNREFUSED	The server on port 3100 is not running. Start it first.
401 even with a token	Variables not saved, `"` copied with the token, or the wrong user's token.
409 on the very first register	That username was already registered. Use Login, or reset the test database.

Coin balance does not match	Top-ups or unlocks were already sent earlier; the numbers carry on. Reset the database to start over.
Upload returns 404	`film_id` is empty or wrong. Copy the id from "Create film" again.
Upload returns 400	No file selected in the `video` row.

> To start completely over, rebuild `shortreel_e2e_test` from `backend/db/shortreel.sql` (test database only; never run it against the real `shortreel` database).